

Process Discovery and Variant Analysis

A Study on the BPIC-17 Loan Application Event Log

Ahmed Omar✉

✉ ahmed.omar@tum.de

November 20, 2025

Abstract — The aim of this report is to present findings and insights from the analysis of the BPIC-17 event log, which documents the loan application process of a Dutch financial institution. The analysis combines automated process discovery with manual refinement to understand the main behavioral patterns and build an interpretable process model. The Inductive Miner and Heuristics Miner algorithms provide initial process models whose limitations become clear through differences in fitness and precision, motivating a structured refinement procedure. By focusing on frequent activities and removing infrequent paths, the final BPMN model achieves a balanced quality profile with a fitness of 0.87 and a precision of 0.73. The report also explains why pm4py and the Celonis Variant Explorer show different variant counts, pointing out that Celonis reduces repeated activities and therefore reports fewer variants. Overall, the results demonstrate how combining algorithmic discovery with structured manual reasoning leads to a clearer and more realistic process model.

1 Introduction

The event log analyzed in this report originates from the BPI Challenge 2017¹ and documents a loan application process carried out by a Dutch financial institution. It covers the period from 2016-01-01 to 2017-02-01, capturing the full lifecycle of loan applications from submission to final decision. The log includes both internal processing steps and interactions with customers.

The report builds on a structured series of steps to analyze and model this process. In Section 3, I begin with a basic examination of the event log, including descriptive statistics and a small filtering step to remove cases without a clear end state. Section 4 applies two process discovery algorithms—Inductive Miner and Heuristics Miner—to obtain initial models and evaluate them using standard pm4py quality metrics. These models serve as a starting point for a manual refinement procedure, supported by throughput-time insights from Celonis and focused on identifying essential activities, removing infrequent behavior, and selecting meaningful loops. This refinement leads to a BPMN model that balances fitness, precision, and simplicity, replaying about 87% of all cases. Section 5 then analyzes why pm4py and the Celonis Variant Explorer report different variant counts and shows that the two tools handle repeated activities differently.

Overall, the report demonstrates how automated discovery and manual analysis can be combined to build a process model that is both realistic and easy to interpret.

2 Technical Setup and Preliminaries

All analyses were conducted in a Jupyter Notebook environment using Python 3.13.5. The pm4py library was used for process discovery, conformance checking, and the computation of quality metrics [1]. Data preprocessing and the computation of basic event log statistics were performed with pandas. For visual inspection of throughput times and activity relations, the commercial process mining tool Celonis was used [2]. The final BPMN model was created using bpmn.io. ChatGPT was used for writing support. All code and the BPMN models used in this analysis are available at: <https://github.com/akado587/Process-Discovery-and-Variant-Analysis>

¹<https://ais.win.tue.nl/bpi/2017/challenge.html>

3 Basic Event Log Analysis

This section provides an overview of the main characteristics of the event log. The following tables present the basic statistics, including the number of cases, events, activities, and attributes, together with descriptive metrics on case lengths and case durations.

Table 1 Basic Event Log Statistics

Metric	Value
Number of events	1,202,267
Number of cases	31,509
Number of activities	26
Number of process variants	15,930
Number of case labels	4
Number of event labels	15
Number of originators in the data	149
Number of categorical event attributes	9

Table 2 Case Length Statistics

Metric	Value
Mean case length	38.16
Median case length	35
Standard deviation	16.72

Table 3 Case Duration Statistics

Metric	Value
Mean duration	21 days, 21 hours, 35 minutes, 25 seconds
Median duration	19 days, 2 hours, 6 minutes, 20 seconds
Standard deviation	13 days, 4 hours, 3 minutes, 41 seconds

Following the basic analysis, I applied a small preprocessing step inspired by the filtering procedure used in the academic winner report [3] of the BPI Challenge 2017. In their report, the authors remove cases that do not show a clear end state, as these cases may still have been in progress at the moment the log was extracted. Based on this idea, I filtered the log to keep only cases that contain one of the final application decisions: *A_Pending*, *A_Cancelled*, or *A_Denied*. These activities indicate that the case reached a meaningful outcome. The original log contains 31,509 cases. After this filter, 31,411 cases remain, so 98 cases were removed.

4 Process Discovery and Model Refinement

This part describes how the process models were discovered, analyzed, and refined. The goal was to understand the real process flow, evaluate the automatically generated models, and create a final BPMN model that combines quality with simplicity. I started this part by applying two process discovery algorithms — the Inductive Miner [4] and the Heuristics Miner [5] — to the event log. Both models were converted into Petri nets and evaluated using the standard *pm4py* [1] quality metrics: fitness, precision, generalization, and simplicity.

The Inductive Miner produced a model with perfect fitness but very low precision, meaning that it allowed a lot of behavior not actually observed in the log. The Heuristics Miner gave more realistic results with a slightly lower fitness but much better precision. These initial models gave me a first understanding of the overall

process and its main behavioral patterns. Figure 1 provides an overview of the refinement steps that led from the discovered models to the final BPMN model.

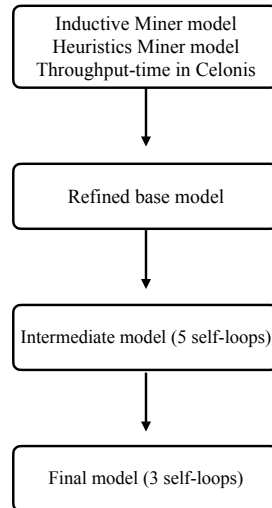


Figure 1 Overview of the refinement steps from the discovered models to the final BPMN model.

To build the final BPMN model, I looked at the most frequent activities in the event log — selecting the 20 most common ones out of 26 — to focus on the main process behavior. Using *Celonis* [2], I examined the average throughput time between consecutive activities, which helped me identify and group related tasks. Figure 2 shows the throughput-time analysis used for identifying these task groups.

Based on these observations, I performed a manual refinement step focusing on removing rarely occurring paths. The outcome of this step was a base model that represents the core process behavior. Figure 3 shows this base model.

4.1 Loop Analysis and Model Adjustments

When I evaluated the refined base model in Figure 3 with alignment-based fitness, it reproduced only about 0.63 of the cases. The precision, however, was relatively high 0.73 , which indicated that the model was still too restrictive and missing important behavior. This led me to compare my model more closely with the discovered models and with the Celonis process view.

During this comparison, I found several self-loops that occurred frequently in the log but were missing in my model:

- *W_Handle leads* → *W_Handle leads*
- *W_Complete application* → *W_Complete application*
- *W_Call after offers* → *W_Call after offers*
- *W_Call incomplete files* → *W_Call incomplete files*
- *W_Validate application* → *W_Validate application*

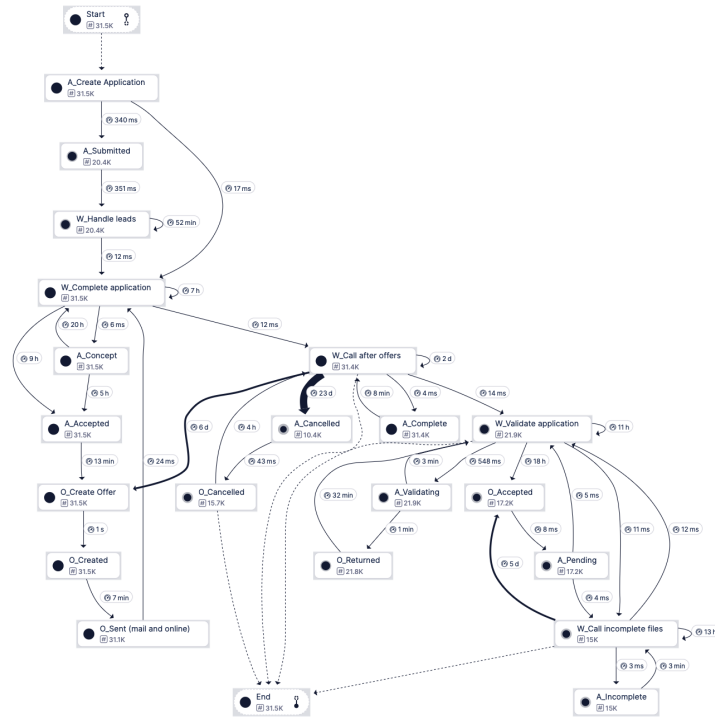


Figure 2 Average throughput time between consecutive activities in Celonis.

I added all five loops to the base model to evaluate their impact. As expected, the fitness increased significantly (to around 0.94), because loops allow the model to reproduce repeated behavior. However, the precision dropped to about 0.55. This showed that the model had become too flexible.

To find a better balance, I systematically tested all loop combinations. The best trade-off was achieved by keeping the three loops

- *W_Call after offers* → *W_Call after offers*,
- *W_Call incomplete files* → *W_Call incomplete files*, and
- *W_Validate application* → *W_Validate application*,

while omitting the loops on *W_Handle leads* and *W_Complete application*. These three loops were crucial for achieving a good fitness value, but they did not reduce precision in the same way the other loops did.

With these three loops included, the model reached the following quality metrics:

- **Fitness:** 0.87
- **Precision:** 0.73
- **Generalization:** 0.99
- **Simplicity:** 0.66

Compared to the models generated directly by the discovery algorithms, the final version shows a better balance between accuracy and simplicity while still replaying around 87% of all cases. Figure 4 shows the complete comparison of all discovered and refined models.

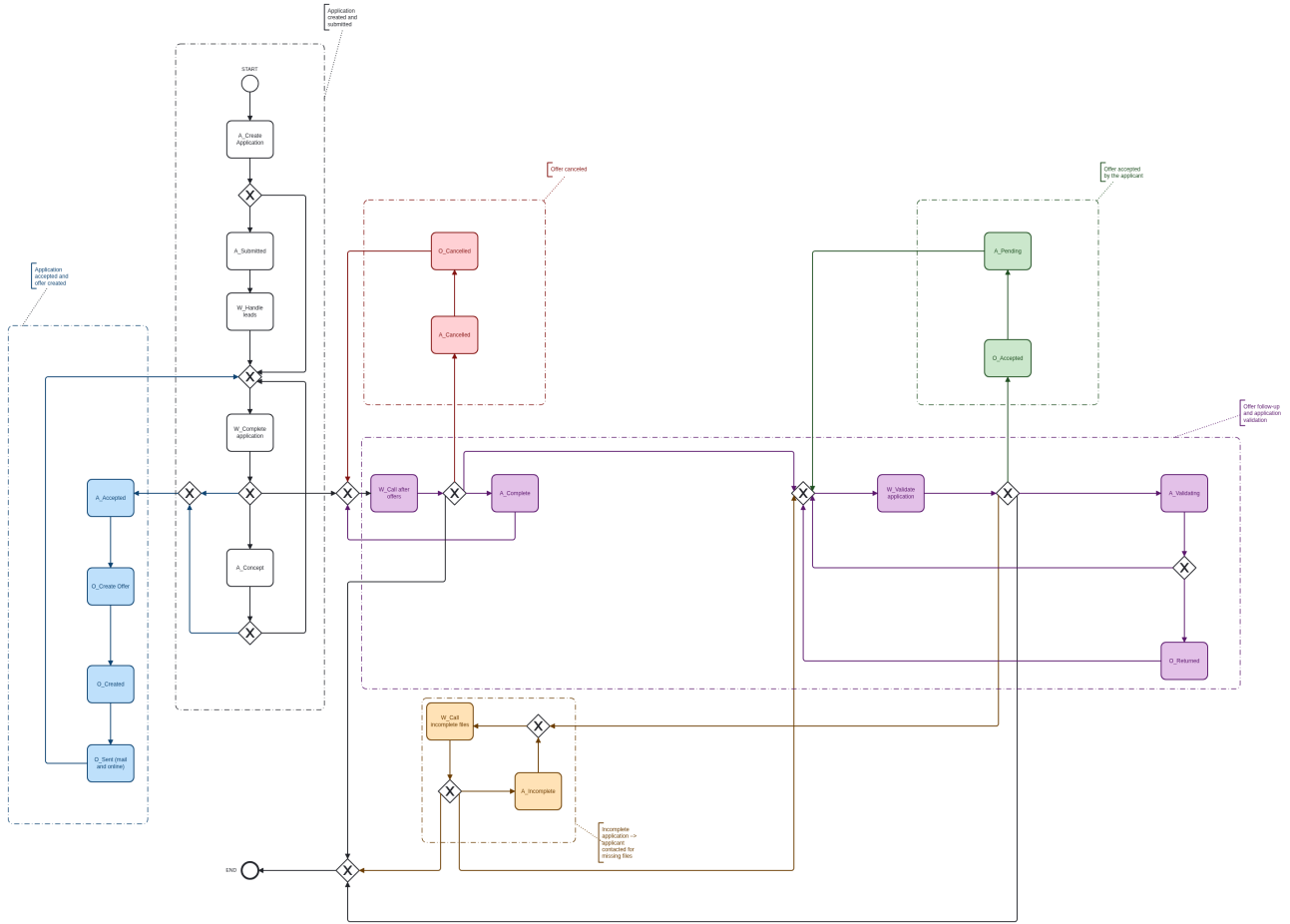


Figure 3 BPMN model after manual refinement. Colors represent grouped tasks identified through throughput-time analysis.

4.2 Custom Simplicity Metrics

In addition to the built-in simplicity metric from *pm4py*, I implemented three structural metrics to evaluate the model structure, following the definitions in [6]. These are:

- **Size:** total number of nodes and arcs
- **Connectivity:** ratio between arcs and nodes
- **Density:** ratio between existing arcs and the maximum possible number of arcs

4.3 Summary

Overall, this part of the work showed me that current process discovery algorithms still have clear limitations. Their results are useful as a starting point but not reliable enough on their own. By comparing and refining them manually, I learned that combining automated discovery with human understanding can lead to a much clearer and more realistic process model.

Model	Fitness	Precision	Generalization	Simplicity (pm4py)	Custom Simplicity Metrics
Alpha Miner model	0.24	0.09	0.98	0.58	Size : 90 Connectivity: 1.37 Density: 0.04
Inductive Miner model	1.0	0.16	0.94	0.63	Size : 340 Connectivity: 1.30 Density: 0.01
Heuristics Miner model	0.95	0.67	0.93	0.52	Size : 295 Connectivity: 1.46 Density: 0.01
Refined base model	0.63	0.73	0.99	0.69	Size : 120 Connectivity: 1.22 Density: 0.02
Intermediate model (5 self-loops)	0.94	0.55	0.99	0.64	Size : 139 Connectivity: 1.28 Density: 0.02
Final model (3 self-loops)	0.87	0.73	0.99	0.66	Size : 129 Connectivity: 1.26 Density: 0.02

Figure 4 Comparison of quality metrics across process discovery algorithms and the refined model.

5 Comparison of Variant Calculation between pm4py and Celonis

5.1 Hypothesis

The analysis started with a clear discrepancy in the number of process variants reported by the two tools. pm4py returned 15,930 variants, while the Celonis Variant Explorer showed 7,080 variants. Since both results were based on the same event log, such a large difference required further investigation.

Initial Observation:

- pm4py: 15,930 variants
- Celonis Variant Explorer: 7,080 variants
- Discrepancy: factor 2.25

Based on this, the working hypothesis was that both tools apply different variant calculation methods, especially regarding how consecutive duplicate activities (self-loops) are handled.

5.2 Approach and Result

I compared the variant counts using three steps:

1. Initial verification:

I confirmed that both tools processed the same event log, used the same activity attribute (concept : name), and had the same number of cases (31,509). No filtering or lifecycle reduction was applied by default.

2. Self-loop investigation:

Many traces in the log contain repeated consecutive activities. To understand their effect on the variant count, I tested how the variant count changes when these repetitions are simplified. In pm4py, a full removal of consecutive duplicates reduced the number of variants from 15,930 to 4,290. This showed that repeated activities have a strong impact on the total count.

3. Tool documentation review:

I checked the Celonis documentation for its variant logic. Celonis uses a “SHORTENED” variant representation by default, where repeated activities are limited to a maximum of two consecutive occurrences ($\text{max_cycle_length} = 2$)².

To validate this behavior, I implemented the same shortening logic in Python and recalculated the variants. This resulted in **7,084 variants**, which matches the Celonis value (displayed as “7.08k”) except for a small rounding difference. Figure 5 shows the number of variants reported by the Celonis Variant Explorer.

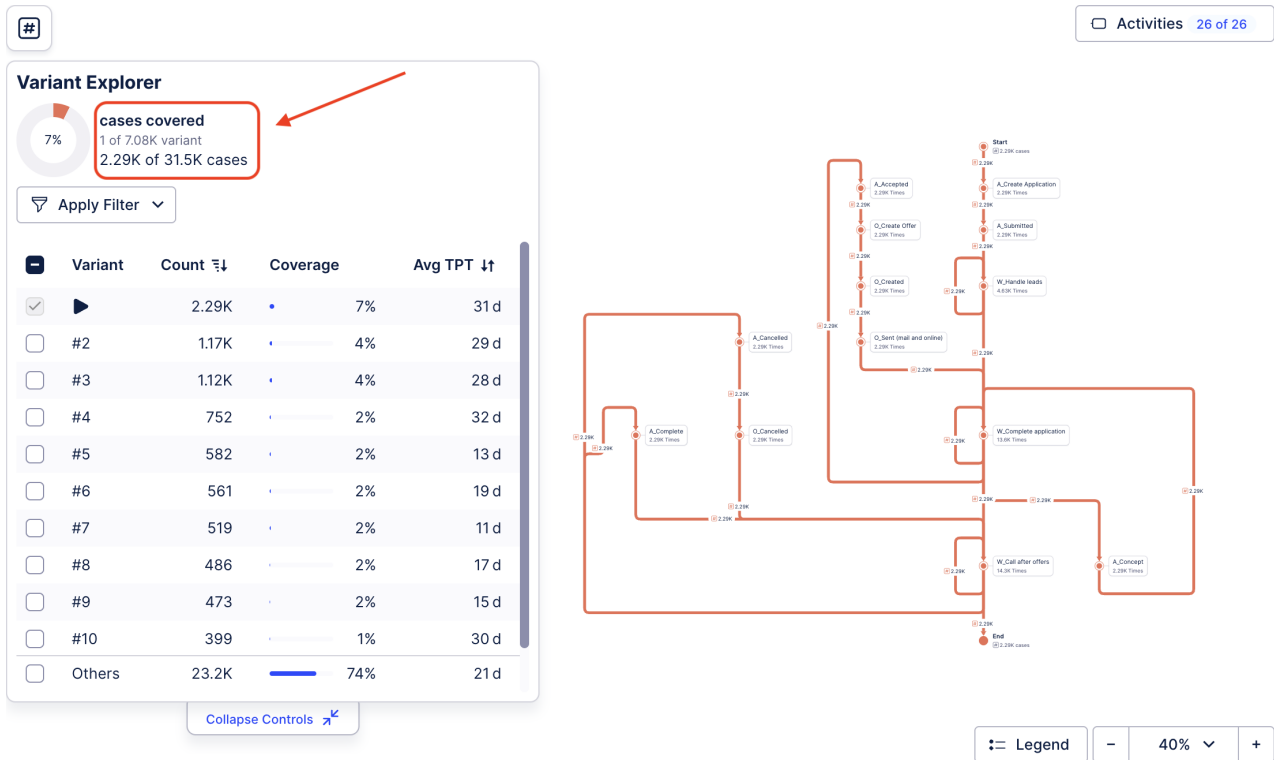


Figure 5 Variant count displayed in the Celonis Variant Explorer.

Result summary:

Method	Variants
pm4py (raw)	15,930
pm4py (max_cycle_length=2)	7,084
Celonis Variant Explorer	~7,080
pm4py (full deduplication)	4,290

Conclusion:

The discrepancy is therefore fully explained by the default Celonis shortening method.

5.3 Interpretation

The shortened variants used by Celonis provide a more compact view of the process. They preserve the main control-flow structure while reducing noise from repeated activities. From a modeling perspective, this is useful: prediction models benefit from more stable variant categories, and simulation models typically separate process structure from the probability of repeated steps. A shortened representation therefore offers a clearer basis for further modeling, while details on repetitions can still be captured separately if needed.

²<https://docs.celonis.com/en/variant-explorer>

References

- [1] Alessandro Berti, Sebastiaan J. van Zelst, and Wil M. P. van der Aalst. PM4Py: A process mining library for Python. *SoftwareX*, 23:101352, 2023.
- [2] Celonis process mining. <https://www.celonis.com/>.
- [3] Ariane Rodrigues, Cassio Almeida, Daniel Saraiva, Felipe Moreira, Georges Spyrides, Guilherme Varela, Gustavo Krieger, Igor Peres, Leila Dantas, Mauricio Lana, Odair Alves, Rafael França, Ricardo Neira, Sonia Gonzalez, William Fernandes, Simone Barbosa, Marcus Poggi, and Hélio Lopes. Stairway to value: Mining the loan application process. Technical report, Pontifícia Universidade Católica do Rio de Janeiro (PUC-Rio), 2017. Academic Winner Report, BPI Challenge 2017.
- [4] Sander J. J. Leemans, Dirk Fahland, and Wil M. P. van der Aalst. Discovering block-structured process models from event logs. In *Business Process Management*, volume 8094 of *Lecture Notes in Computer Science*, pages 311–329. Springer, 2013.
- [5] A. J. M. M. Weijters, Wil M. P. van der Aalst, and A. K. Alves de Medeiros. Process mining with the HeuristicsMiner algorithm. BETA Working Paper Series WP 166, Eindhoven University of Technology, Eindhoven, 2006.
- [6] Josep Carmona, Boudewijn F. van Dongen, Andreas Solti, and Matthias Weidlich. *Conformance Checking: Relating Processes and Models*. Springer, 2018.